

Unidad V

Orientación a Objetos (OO)



Orientación a Objetos

- Características de OO

- Identidad.
- Abstracción.
- Clasificación.
- Encapsulamiento.
- Herencia.
- Polimorfismo.
- Persistencia.

•Identidad: los datos son organizados en entidades discretas, distinguibles, denominadas objetos. Un objeto simple tiene estados y comportamiento asociados. Cada objeto en un sistema OO usualmente tiene un nombre, que lo distingue de otro objeto.

•Abstracción: Descripción simplificada del sistema que resalta unos detalles y suprime otros. Es esencial para construir cualquier sistema, sea orientado a objetos o no.

•Clasificación: agrupa objetos que tiene atributos y comportamientos en común

•Encapsulamiento: Ocultar detalles de un objeto que no contribuyen a sus características esenciales. Una clase encapsula los atributos y comportamientos de un objeto, ocultando los detalles de implementación.

•Herencia: Abstracciones organizadas por niveles.

•Polimorfismo: Esta característica permite definir distintos comportamientos para un método dependiendo de la clase sobre la que se realice la implementación. En todo momento tenemos un único medio acceso, sin embargo se podrá acceder a métodos distintos.

•Persistencia: capacidad del nombre, estados y comportamientos de un objeto para trascender el espacio o tiempo, es decir, se conservan cuando el objeto es transformado.

Orientación a Objetos

- POO
 - **Objeto**
 - Definido por una lista de atributos y una lista de procedimientos que se pueden llevar a cabo sobre esos atributos.
 - **Método**
 - Se define como un procedimiento o una función que altera el estado de un objeto y/o hace que el objeto envíe un mensaje.
 - **Clase**
 - Colección de objetos que comparten atributos y métodos comunes. Se puede considerar como una plantilla para crear instancias.

•Un **Objeto** está definido por una lista de atributos (con frecuencia, divididos en variables de instancia y variables de clase), y una lista de procedimientos que se pueden llevar a cabo sobre esos atributos. En la programación orientada a objetos, las variables de instancia no son visibles para otros objetos.

Una variable de instancia es una variable que se relaciona con una única instancia de una clase.

Cada vez que se crea un objeto, el sistema crea una copia de todas las variables que están vinculadas con dicha clase, haciéndolas propias de esa instancia. Solamente se puede acceder a ellas a través del identificador del objeto.

Estas variables son declaradas fuera del cuerpo de los métodos y dentro de la clase, por lo tanto son de tipo global. Por ende, pueden ser utilizadas por cualquier método no estático de dicha clase.

Una variable de clase es una variable, al contrario que las variable de instancia, propia de la clase que la contiene, y no de instancias de la misma, esto quiere decir que todos los objetos que se creen de esta clase, comparten su valor, son llamadas variables estáticas.

•Un **método** se define como un procedimiento o una función que altera el estado de un objeto y/o hace que el objeto envíe un mensaje, es decir, que devuelva valores. Los métodos se ejecutan en respuesta a mensajes y manejan los valores de los datos del objeto al cual pertenecen.

•Una **clase** es una colección de objetos que comparten atributos y métodos comunes. Se puede considerar como una plantilla para crear instancias. Cada objeto es una instancia o caso particular de una clase y se diferencia de los otros objetos de su clase por el valor de sus atributos y por su identificación.

Clases: Una clase es la unidad básica que encapsula la descripción abstracta de una agrupación de objetos y define los atributos y los métodos de esa serie de objetos. Todos los objetos de esta clase (instancias de esa clase) tienen el mismo comportamiento y el mismo conjunto de atributos (cada objeto el suyo propio).

Orientación a Objetos

- POO
 - **Encapsulamiento (ocultamiento de información)**
 - Práctica de incluir dentro de un objeto todo lo que necesita, y de hacerlo además, de tal manera que ningún otro objeto necesite conocer nunca su estructura interna.
 - **Polimorfismo (sobrecarga de operadores)**
 - Posibilidad de que una variable o función adopte distintas formas en tiempo de ejecución o, más específicamente, a la posibilidad de referirse a instancias de varias clases.
 - **Mensaje**
 - Consiste en una dirección y una instrucción formada por un nombre o método y cero o más parámetros.

•El **encapsulamiento** (ocultamiento de información) se refiere a la práctica de incluir dentro de un objeto todo lo que necesita, y de hacerlo además, de tal manera que ningún otro objeto necesite conocer nunca su estructura interna. La realización del almacenamiento de datos siempre es privada para el objeto.

•El **polimorfismo** (sobrecarga de operadores) se refieren a la posibilidad de utilizar el mismo símbolo con fines distintos. El polimorfismo se refiere a la posibilidad de que una variable o función adopte distintas formas en tiempo de ejecución o, más específicamente, a la posibilidad de referirse a instancias de varias clases. La sobrecarga es un caso especial en el que dos operaciones a instancias comparten simplemente el mismo nombre.

•Un **mensaje** consiste en una dirección y una instrucción formada por un nombre o método y cero o más parámetros. Si el destinatario contiene un método para el que la instrucción tiene sentido, entonces devuelve una respuesta al objeto emisor, sino devuelve un mensaje de error. La estructura de un mensaje podría ser la siguiente: **objeto método [parámetros]**, o también: **objeto.método(parámetros)**.

Orientación a Objetos

Una **clase** es la unidad básica que encapsula la descripción abstracta de una agrupación de objetos y define los atributos y los métodos de esa serie de objetos. Todos los objetos de esta clase (instancias de esa clase) tienen el mismo comportamiento y el mismo conjunto de atributos (cada objeto el suyo propio).

Coche
+propietario: string
+modelo: string
+precio: int
+cambiarPropietario(): void

A modo de ejemplo podríamos definir una clase llamada Coche para representar de forma abstracta todos los objetos coche, un objeto coche es la representación concreta de la clase Coche, así tendríamos por ejemplo un objeto coche con el propietario Luis, modelo BMW y precio 50.000.

Orientación a Objetos

Lo normal para representar los **atributos** de una clase es mostrar su nombre y valor, pero también puede mostrarse su valor inicial, privacidad (+ Indica atributos públicos; # Indica atributos protegidos; - Indica atributos privados) u otras propiedades.

Las **operaciones** o **métodos** también se muestran al menos con su nombre, pudiendo mostrar sus parámetros y valores de retorno, así como su privacidad.

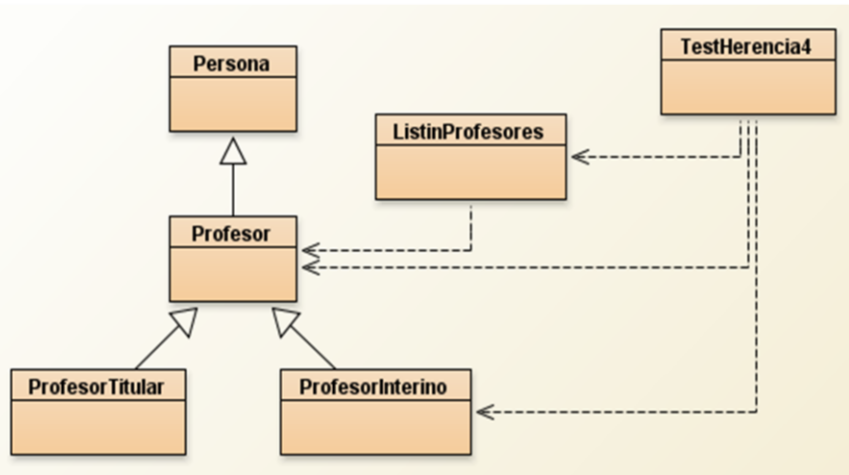
Nombre de la clase
Nombre_atributo_1: tipo_de_dato_1 = valor_por_defecto_1
Nombre_atributo_2: tipo_de_dato_2 = valor_por_defecto_2
Nombre_operacion_1(lista_de_parámetros_1): tipo_resultado_1

Nombre de la clase
+Atributo público
#Atributo protegido
-Atributo privado
+Operación pública()
#Operación protegida()
-Operación privada()

En el ejemplo del coche, tendríamos como atributos de la clase Coche: propietario, modelo y precio.

En el ejemplo del coche, tendríamos como operaciones o métodos, una sola operación, cambiarPropietario() que será un método que servirá para cambiar el atributo propietario de la clase Coche.

Orientación a Objetos



Dependencia o Instanciación (uso):

Representa un tipo de relación muy particular, en la que una clase es instanciada (su instanciación es dependiente de otro objeto/clase). Se denota por una flecha punteada.

El uso más particular de este tipo de relación es para denotar la dependencia que tiene una clase de otra, como por ejemplo una aplicación grafica que instancia una ventana (la creación del Objeto Profesor esta condicionado a la instanciación proveniente desde el objeto ListenProfesores):

Cabe destacar que el objeto creado (en este caso la Profesor) no se almacena dentro del objeto que lo crea (en este caso la Aplicación).

Orientación a Objetos

• Diseño de OO

- Se deben identificar objetos y describir sus interrelaciones.
- Casos de uso
- UML (Lenguaje unificado de modelado)
- Sustantivos -> Objetos
- Verbos -> Comportamientos
- Diseño de la gestión de tareas
 - Tipos:
 - Dirigida por evento
 - Dirigida por tiempo

•Identificar objeto y describir sus interrelaciones

•Casos de uso

- Describe una funcionalidad particular que se supone que el sistema debe realizar o exhibir, modelando el diálogo que un usuario, un sistema externo u otra entidad tendrán con el sistema a ser desarrollado.
- Útiles para comunicarse con clientes, diseñadores y probadores de sistemas.

•UML (Lenguaje unificado de modelado)

- Puede utilizarse para visualizar, especificar o documentar un problema. Incluyen la vista dinámica del sistema, la vista dinámica (casos de usos, lista de actividades, diagramas de interacciones) y estática del sistema (diagrama de clases).

•Sustantivos -> Objetos

- Estructuras
- Sistemas externos
- Dispositivos
- Roles
- Procedimientos operativos
- Lugares
- Organizaciones
- Cosas que son manipuladas por el sistema

•Verbos -> Comportamientos

- Verbos imperativos
- Verbos pasivos
- Acciones
- Cosas o eventos a recordar.

•Diseño de la gestión de tareas

•Tipos:

- Dirigida por evento (se inicia cuando ocurre algún evento en particular)
- Dirigida por tiempo (se invoca en instantes particulares de tiempo)

•Proceso:

- Identificar las tareas y clasificarlas.
- Determinar prioridades (si las 2 son invocadas simultáneamente).

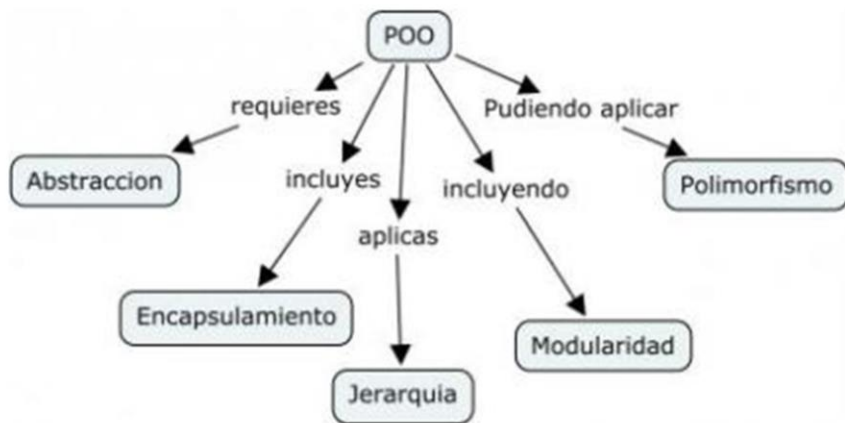
- Crear una tarea para coordinar las demás.
- Diseñar objetos para cada tarea y definir las relaciones entre ellos.

Orientación a Objetos

- **Estrategia de diseño:**

- Antes de resolver el diseño es necesario tomar decisiones generales sobre las estrategias de diseño a seguir.
- Algunas de las decisiones a tomar se presentan a continuación y se relacionan con aspectos que incluyen la arquitectura, la robustez, reuso y extensibilidad del sistema

Orientación a Objetos



Orientación a Objetos

- **Arquitectura**

- Se refiere a la organización de las clases dentro del sistema.
- Como parte de la arquitectura de diseño se decide cómo distribuir la inteligencia entre las clases y qué aspectos de la inteligencia del sistema se debe asignar a cada una de ellas.
- Para ello existen tres alternativas principales

Arquitectura. Se refiere, en este caso, a la organización de las clases dentro del sistema.

Durante el modelo de análisis se generó una arquitectura de clases para el sistema y se definió la funcionalidad conceptual ofrecida por las distintas clases dentro de la arquitectura.

Durante el diseño debe detallarse esta arquitectura, pudiéndose cambiar los aspectos considerados inicialmente, como fue la funcionalidad asignada a cada clase, e incluso las propias clases.

Como parte de la arquitectura de diseño se decide cómo distribuir la inteligencia entre las clases y qué aspectos de la inteligencia del sistema se debe asignar a cada una de ellas.

Para ello existen tres alternativas principales...

Orientación a Objetos

• Arquitectura – Primer Enfoque

- Minimizar el número de clases inteligentes
- En el caso más extremo, sólo un objeto tendría conocimiento sobre todo el sistema. Los demás objetos tendrán un mínimo de inteligencia y el objeto inteligente servirá como controlador de los demás.
- Ventaja: sólo se requerirá comprender el flujo de control dentro del objeto principal para entender toda la aplicación
- Desventaja: se vuelve más compleja la extensibilidad del sistema, ya que cualquier cambio en el flujo de control se llevaría a cabo en un mismo objeto y afectaría potencialmente la lógica de toda la aplicación

Un **primer enfoque** es minimizar el número de clases inteligentes.

En el caso más extremo, sólo un objeto tendría conocimiento sobre todo el sistema. Los demás objetos tendrán un mínimo de inteligencia y el objeto inteligente servirá como controlador de los demás.

Una ventaja de este enfoque es que sólo se requerirá comprender el flujo de control dentro del objeto principal para entender toda la aplicación.

Sin embargo, se vuelve más compleja la extensibilidad del sistema, ya que cualquier cambio en el flujo de control se llevaría a cabo en un mismo objeto y afectaría potencialmente la lógica de toda la aplicación.

De cierta manera, esto se considera como la “estructuración” del programa, en otras palabras, **se transforma la orientación a objetos a programación estructurada**, donde toda la aplicación consta de un solo “objeto”.

Orientación a Objetos

• Arquitectura – Segundo Enfoque

- La inteligencia del sistema lo más homogéneo posible, diseñando todas las clases con inteligencia similar
- Ventaja:
 - La distribución de la inteligencia del sistema de manera homogénea entre los objetos permite que cada objeto sepa menos cosas. Esto produce objetos más pequeños y más fáciles de comprender
- Desventajas:
 - Una distribución perfectamente homogénea es una tarea casi imposible, ya que los objetos varían en sus responsabilidades
 - Si todas las clases son “inteligentes”, esto significaría que éstas serán muy especializadas, dificultando la extensibilidad del sistema que requiere generalizar más las clases.

Un **segundo enfoque** es distribuir la inteligencia del sistema lo más homogéneo posible, diseñando todas las clases con inteligencia similar. **Este enfoque se aproxima más con el espíritu de la orientación a objetos.**

Sin embargo, una distribución perfectamente homogénea es una tarea casi imposible, ya que los objetos varían en sus responsabilidades, su razón de ser depende de la aplicación.

Por otro lado, la distribución de la inteligencia del sistema de manera homogénea entre los objetos permite que cada objeto sepa menos cosas. Esto produce objetos más pequeños y más fáciles de comprender.

La desventaja es que la inteligencia del sistema va de la mano con la especialización de las clases.

Si todas las clases son “inteligentes”, esto significaría que éstas serán muy especializadas, dificultando la extensibilidad del sistema que requiere generalizar más las clases.

Orientación a Objetos

• Arquitectura – Tercer Enfoque

- Encontrar un balance entre los dos primeros.
- La idea es homogeneizar la inteligencia del sistema sólo en ciertas clases, como las de control.
- El resto de las clases, entidad y borde, serán “tontas”, generalmente sobreviviendo a modificaciones en el sistema y manteniendo la lógica introducida durante el modelo de requisitos y el modelo de análisis posterior para lograr una mayor robustez del sistema.

• Modelo MVC

Un **tercer enfoque** es encontrar un balance entre los dos primeros. La idea es homogeneizar la inteligencia del sistema sólo en ciertas clases, como las de control.

El resto de las clases, entidad y borde, serán tontas, generalmente sobreviviendo a modificaciones en el sistema y manteniendo la lógica introducida durante el modelo de requisitos y el modelo de análisis posterior para lograr una mayor robustez del sistema.

MVVM significa Modelo Vista Vista Modelo, porque en este patrón de diseño se separan los datos de la aplicación, la interfaz de usuario pero en vez de controlar manualmente los cambios en la vista o en los datos, estos se actualizan directamente cuando sucede un cambio en ellos, por ejemplo si la vista actualiza un dato que está presentando se actualiza el modelo automáticamente y viceversa.

El Patrón Modelo-Vista-Presentador (MVP) surge para ayudar a realizar pruebas automáticas de la interfaz gráfica, para ello la idea es codificar la interfaz de usuario lo más simple posible, teniendo el menor código posible, de forma que no merezca la pena probarla. En su lugar, toda la lógica de la interfaz de usuario, se hace en una clase separada (que se conoce como Presentador), que no dependa en absoluto de los componentes de la interfaz gráfica y que, por tanto, es más fácil de realizar pruebas.

La idea básica es que la clase Presentador haga de intermediario entre la Vista (la interfaz gráfica de usuario) y el modelo de datos. La vista tiene métodos en los que le pasan los datos que debe pintar ya “mascados” (una lista de cadenas por ejemplo, en vez del modelo...). Únicamente debe meter

esos datos en los componentes gráficos (cajas de texto, checkbox, etc). También métodos get para obtener el contenido de esos componentes.

Orientación a Objetos

El modelo–vista–controlador (MVC) es un patrón de arquitectura de software, que separa los datos y la lógica de negocio de una aplicación de la interfaz de usuario y el módulo encargado de gestionar los eventos y las comunicaciones.

Para ello MVC propone la construcción de tres componentes distintos que son el modelo, la vista y el controlador, es decir, por un lado define componentes para la representación de la información, y por otro lado para la interacción del usuario.

Este patrón de arquitectura de software se basa en las ideas de reutilización de código y la separación de conceptos, características que buscan facilitar la tarea de desarrollo de aplicaciones y su posterior mantenimiento.

Orientación a Objetos

Es un patrón de arquitectura de las aplicaciones software

- Separa la lógica de negocio de la interfaz de usuario
- Facilita la evolución por separado de ambos aspectos
- Incrementa reutilización y flexibilidad

Modelo-Vista-Controlador

- Un modelo
- Varias vistas
- Varios controladores
- Las vistas y los controladores suelen estar muy relacionados
 - Los controladores tratan los eventos que se producen en la interfaz gráfica (vista)
- Esta separación de aspectos de una aplicación da mucha flexibilidad al desarrollador

Orientación a Objetos

Flujo de control

1. El usuario realiza una acción en la interfaz
2. El controlador trata el evento de entrada.
3. El controlador notifica al modelo la acción del usuario, lo que puede implicar un cambio del estado del modelo (si no es una mera consulta)
4. Se genera una nueva vista. La vista toma los datos del modelo.
El modelo no tiene conocimiento directo de la vista
5. La interfaz de usuario espera otra interacción del usuario, que comenzará otro nuevo ciclo.

Orientación a Objetos

MCV EN APLICACIONES WEB

- Vista:
 - La página HTML
- Controlador:
 - Código que obtiene datos dinámicamente y genera el contenido HTML
- Modelo:
 - La información almacenada en una base de datos o en XML
 - Junto con las reglas de negocio que transforman esa información (teniendo en cuenta las acciones de los usuarios)

Orientación a Objetos

MCV EN JAVA

- Modelo:
 - El modelo lo realiza el desarrollador
- Vista:
 - Conjunto de objetos de clases que heredan de `java.awt.Component`
- Controlador:
 - El controlador es el thread de tratamiento de eventos, que captura y propaga los eventos a la vista y al modelo
 - Clases de tratamiento de los eventos (a veces como clases anónimas) que implementan interfaces de tipo `EventListener` (`ActionListener`, `MouseListener`, `WindowListener`,)

Orientación a Objetos

Ejemplo:

Calculadora de euros a dólares

- Una clase sencilla que da operaciones para calcular la conversión entre euros y dólares
- En el constructor se indica el cambio
- Métodos
 - Calcular dólares de una cantidad en euros
 - Calcular euros de una cantidad en dólares

Orientación a Objetos

- **Robustez**

- El sistema debe ser protegidos contra errores y ofrecer diagnósticos que permitan identificar fallas, en particular aquellas que son fatales.
- Debe ser uno de los objetivos principales del diseño.
- Las principales consideraciones relacionadas con la robustez de un sistema son las siguientes:
 - El sistema debe estar protegido contra parámetros incorrectos proporcionados por el usuario.
 - El encapsulamiento es fundamental para la robustez del sistema. Únicamente la interfaz de los métodos afecta a las demás clases.

Robustez. La robustez de un sistema debe ser uno de los objetivos principales del diseño.

El sistema debe ser protegidos contra errores y ofrecer diagnósticos que permitan identificar fallas, en particular aquellas que son fatales.

Durante el desarrollo, a veces es bueno insertar instrucciones internas en el código para descubrir fallas, aunque luego se eliminan durante la producción.

En general, se debe escoger lenguajes de programación que apoyen estos aspectos, como son el manejo de excepciones.

Orientación a Objetos

- **Reuso**

- Cuanto más se pueda reutilizar el código será mejor la robustez del sistema.
- Algunas estrategias para mejorar las posibilidades de reuso del diseño:
 - A través de la herencia se incrementa el reuso del código. Se toman los aspectos comunes a las clases similares utilizando superclases comunes.
 - La delegación provee un mecanismo para el reuso de código, pero sin utilizar herencia. Esto se basa en el uso de agregación a través de clases intermediarias que ocultan la funcionalidad de las clases a las cuales se delega.
 - El encapsulamiento es muy efectivo para lograr el reuso, se aplica tanto al nivel de los objetos como de componentes desarrollados en otras aplicaciones. Estos componentes se reutilizan como se diseñaron, agregándolos a nuevas interfaces.

Reuso. El reuso es un aspecto fundamental del diseño. Cuanto más se pueda reutilizar el código será mejor la robustez del sistema. Las siguientes son algunas estrategias para mejorar las pos

- A través de la herencia se incrementa el reuso del código. Se toman los aspectos comunes a las clases similares utilizando superclases comunes. Este enfoque es efectivo cuando las diferencias entre las clases son pequeñas y las similitudes son grandes. Es importante considerar la naturaleza de cada herencia para asegurar que no se llegue a extremos donde la aplicación de la herencia sea inadecuada.

Orientación a Objetos

- **Extensibilidad**

- La mayor parte de los sistemas son extendidos de manera imprevista.
- Las siguientes son algunas de las perspectivas de extensibilidad:
 - Se debe encapsular otra vez las clases, ocultando su estructura interna a las otras clases. Sólo los métodos de las clases deben acceder sus atributos.
 - Una clase particular debe tener un conocimiento limitado de la arquitectura de clases del sistema. Este conocimiento abarcará únicamente las asociaciones entre éstas y sus clases vecinas directas.
 - Se deben evitar expresiones que requieran un conocimiento explícito de los tipos de objetos. En su lugar debe aprovechar el polimorfismo.
 - Se debe distinguir entre operaciones privadas y públicas. Las operaciones privadas pueden modificarse sin afectar a otras clases.

Extensibilidad. La mayor parte de los sistemas son extendidos de manera imprevista. Las siguientes son algunas de las perspectivas de extensibilidad:

- Se debe encapsular otra vez las clases, ocultando su estructura interna a las otras clases. Sólo los métodos de las clases deben acceder sus atributos.

Unidad IV – Orientación a Objetos

FIN